

**-LAPORAN PEMROGRAMAN BERBASIS OBJECT
JOBSHEET 4**



Nama:Daffa Dwi Nugroho

NIM : 254107020140

Kelas: TI 2-A

Link Github: <https://github.com/Dapa-yap/Praktikum-Berbasis-Objek.git>

D-IV TEKNIK INFORMATIKA

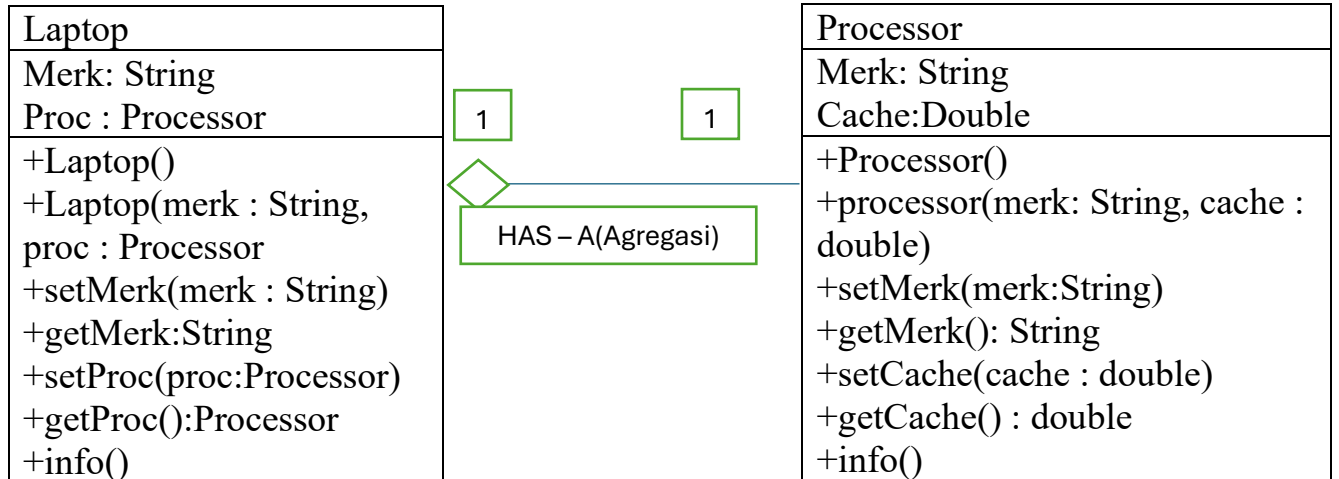
JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN 2026

JOBSHEET 4

Percobaan 1: Aggregation Satu-ke-Satu (Laptop dan Processor)

Langkah 1: Diagram kelas



Langkah 2: Buat class Processor

Buat package id.ac.polinema.relasiclass.percobaan1, lalu buat class Processor dengan dua atribut private berikut

```
package id.ac.polinema.relasiclass.percobaan1;

public class Processor{
    private String merk;
    private double cache;
}
```

Langkah 3: Tambah constructor Processor

Tambahkan constructor default dan constructor berparameter:

```

package id.ac.polinema.relasticlass.percobaan1;

public class Processor{
    private String merk;
    private double cache;

    public Processor(){

    }
    public Processor(String merk, double cache){
        this.merk = merk;
        this.cache = cache;
    }
}

```

Langkah 4: Tambah setter, getter, dan info()

```

}
    public void setMerk(String merk){
        this.merk = merk;
    }
    public String getMerk(){
        return merk;
    }
    public void setCache(double cache){
        this.cache = cache;
    }
    public double getCache(){
        return cache;
    }
    public void info(){
        System.out.printf(format: "Merk Processor = %s\n", merk);
        System.out.printf(format: "Cache Memory: %.2f\n", cache);
    }
}

```

Langkah 5: Buat class Laptop

```

public class Laptop {
    private String merk;
    private Processor proc;
}

```

Langkah 6: Tambah constructor, setter, dan getter Laptop

```

public class Laptop {
    private String merk;
    private Processor proc;

    public Laptop(){

    }

    public Laptop(String merk, Processor proc){
        this.merk = merk;
        this.proc = proc;
    }

    public void setMerk(String merk){
        this.merk = merk;
    }

    public String getMerk(){
        return merk;
    }

    public void setProc(Processor proc){
        this.proc = proc;
    }

    public Processor getProc(){
        return proc;
    }
}

```

Langkah 7: Tambah method info() dengan delegasi

```

public void info(){
    System.out.printf("Merk Laptop = " + merk );
    proc.info();
}

```

Langkah 8: Uji dengan constructor berparameter

```

Run | Debug
public static void main(String[] args) {
    Processor p = new Processor(merk: "Intel i5", cache: 3);
    Laptop l = new Laptop(merk: "Thinkpad", p);
    l.info();
}

```

```

Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 3,00
PS C:\Users\Daffa Dwi\OneDrive\Documents\SMT_3\PR

```

Langkah 9: Uji dengan constructor default dan setter injecti

```

Processor p1 = new Processor();
p1.setMerk(merk: "Intel i5");
p1.setCache(cache: 4);
Laptop l1 = new Laptop();
l1.setMerk(merk: "Thinkpad");
l1.setProc(p1);
l1.info();

```

```

Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 3,00
Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 4,00

```

Langkah 10: Objek Processor dibuat langsung sebagai argumen (anonymous object)

```

Laptop l2 = new Laptop(merk: "Thinkpad",
    new Processor(merk: "Intel i5", cache: 3));
l2.info();

```

```

Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 3,00
Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 4,00
Merk Laptop = ThinkpadMerk Processor = Intel i5
Cache Memory: 3,00

```

Pertanyaan Percobaan 1

1. Di dalam class Processor dan class Laptop, terdapat method setter dan getter untuk masing-masing atributnya. Apakah gunanya method setter dan getter tersebut?

- Method setter digunakan untuk mengisi atau mengubah nilai pada suatu atribut private dari luar class. Method getter digunakan untuk mengambil atau membaca nilai atribut private tersebut agar bisa digunakan atau ditampilkan di luar class

2. Di dalam class Processor dan class Laptop, masing-masing terdapat konstruktor default dan konstruktor berparameter. Bagaimanakah beda penggunaan dari kedua jenis konstruktor tersebut?

-Konstruktor default digunakan untuk membuat (instansiasi) objek baru dengan nilai atribut kosong atau default bawaan sistem, nilainya biasanya akan diisi

belakangan menggunakan setter. Konstruktor berparameter digunakan saat kita ingin langsung memberikan nilai spesifik ke dalam atribut-atribut tersebut bersamaan dengan saat objek pertama kali dibuat.

3. Perhatikan class Laptop, di antara 2 atribut yang dimiliki (merk dan proc), atribut manakah yang bertipe object? Baris kode manakah yang menunjukkan bahwa class Laptop memiliki relasi dengan class Processor?

-Atribut yang bertipe object pada class Laptop adalah proc. Baris kode yang menunjukkan relasi dengan class Processor adalah baris deklarasi atribut: `private Processor proc`.

4. Perhatikan pada class Laptop, apakah guna dari sintaks `proc.info()`?

-Sintaks ini berfungsi untuk memanggil method `info()` milik objek dari class Processor yang tersimpan dalam atribut `proc`.

5. Pada Langkah 8, objek `p` dibuat lebih dulu baru diberikan ke constructor Laptop. Pada Langkah 10, objek Processor dibuat langsung di dalam argumen constructor Laptop (tanpa variabel `p`). Apakah keduanya menghasilkan output yang berbeda? Mengapa?

-Cara pertama: Objek dibuat lalu "dititipkan" ke sebuah variabel (misalnya `p`) dulu, baru variabel itu yang dimasukkan.

-Cara kedua: Objek langsung dibuat di dalam tanda kurung tanpa diberi nama variabel.

6. Secara kode, apakah relasi Laptop-Processor pada percobaan ini termasuk Aggregation atau Composition? Tunjukkan baris kode yang menjadi bukti jawabanmu.

Aggregation

```
}  
public Laptop(String merk, Processor proc){  
    this.merk = merk;  
    this.proc = proc;  
}
```

```
}  
public void setProc(Processor proc){  
    this.proc = proc;  
}
```

7. Andaikan constructor Laptop diubah menjadi seperti berikut, sehingga Processor dibuat sendiri di dalam Laptop, bukan diterima sebagai parameter: `public Laptop (String merk) { this.merk = merk; this.proc = new Processor ("Generic", 1); }` Apakah relasi Laptop-Processor pada versi ini masih

Aggregation? Jelaskan alasannya (jawaban ini akan kita buktikan sendiri lewat kode pada Percobaan 5).

Percobaan 2: Aggregation dengan Relasi Ganda (Rental Mobil)

Langkah 1: Diagram kelas



Langkah 2: Buat class Mobil

```
public class Mobil {
    private String merk;
    private int biaya;

    public Mobil(){

    }

    public void setMerk(String merk){
        this.merk = merk;
    }
    public String getMerk(){
        return merk;
    }
    public void setBiaya(int biaya){
        this.biaya = biaya;
    }
    public int getBiaya(){
        return biaya;
    }
    public int hitungBiayaMobil(int hari){
        return biaya * hari;
    }
}
```

Langkah 3: Buat class Sopir

```
public class Sopir {  
    private String nama;  
    private int biaya;  
  
    public Sopir(){  
  
    }  
    public void setNama(String nama){  
        this.nama = nama;  
    }  
    public String getNama(){  
        return nama;  
    }  
    public void setBiaya(int biaya){  
        this.biaya = biaya;  
    }  
    public int getBiaya(){  
        return biaya;  
    }  
    public int hitungBiayaSopir(int hari){  
        return biaya * hari;  
    }  
}
```

Langkah 4: Buat class Pelanggan (whole untuk Mobil dan Sopir)

```

public class Pelanggan {
    private String nama;
    private Mobil mobil;
    private Sopir sopir;
    private int hari;
    public Pelanggan(){

    }
    public void setName(String nama){
        this.nama = nama;
    }
    public String getName(){
        return nama;
    }
    public void setMobil(Mobil mobil){
        this.mobil = mobil;
    }
    public Mobil getMobil(){
        return mobil;
    }
    public void setSopir(Sopir sopir){
        this.sopir = sopir;
    }
    public Sopir getSopir(){
        return sopir;
    }
    public void setHari(int hari){
        this.hari = hari;
    }
}

```

Langkah 5: Tambah hitungBiayaTotal()

```

public int hitungTotalBiaya(){
    return mobil.hitungBiayaMobil(hari) + sopir.hitungBiayaSopir(hari);
}

```

Langkah 6: Uji di MainPercobaan2

```

public class MainPercobaan2 {
    Run | Debug
    public static void main(String[] args) {
        Mobil m = new Mobil();
        m.setMerk(merk: merk: "Avanza");
        m.setBiaya(biaya: biaya: 350000);

        Sopir s = new Sopir();
        s.setNama(nama: nama: "John Doe");
        s.setBiaya(biaya: biaya: 200000);

        Pelanggan p = new Pelanggan();
        p.setNama(nama: nama: "Jane Doe");
        p.setMobil(m);
        p.setSopir(s);
        p.setHari(hari: hari: 2);

        System.out.println("Biaya Total: " + p.hitungTotalBiaya());
    }
}

```

```

Biaya Total: 1100000

```

Langkah 7: Navigasi rantai objek (a.b().c())

```

System.out.println("Biaya Total: " + p.hitungTotalBiaya());
System.out.println("Merk Mobil: " + p.getMobil().getMerk());

```

```

Biaya Total: 1100000
Merk Mobil: Avanza

```

Pertanyaan Percobaan 2

1. Perhatikan class Pelanggan. Pada baris program manakah yang menunjukkan bahwa class Pelanggan memiliki relasi dengan class Mobil dan class Sopir?

-Relasi ini ditunjukkan pada baris deklarasi atribut di dalam class Pelanggan.java, yaitu pada baris private Mobil mobil; dan private Sopir sopir;. Ini menunjukkan bahwa class Pelanggan memiliki atribut yang merupakan objek dari class Mobil dan Sopir

2. Perhatikan method hitungBiayaSopir pada class Sopir, serta method hitungBiayaMobil pada class Mobil. Mengapa method tersebut harus memiliki argument hari, padahal hari sendiri adalah atribut milik Pelanggan, bukan milik Mobil atau Sopir?

-Method tersebut bertugas menghitung biaya sewa berdasarkan durasi penyewaan dengan cara mengalikan atribut biaya (milik Mobil atau Sopir)

dengan jumlah hari. Karena data hari disimpan di dalam class Pelanggan, nilai tersebut harus dikirimkan atau di-passing sebagai argumen ke dalam method milik Mobil dan Sopir agar proses perkalian dapat dilakukan di dalam method tersebut

3. Perhatikan kode dari class Pelanggan. Untuk apakah perintah `mobil.hitungBiayaMobil(hari)` dan `sopir.hitungBiayaSopir(hari)`?

-Perintah ini digunakan untuk meminta objek Mobil menghitung total biaya sewa mobilnya selama durasi hari tertentu, dan meminta objek Sopir menghitung total biaya sewa jasanya selama durasi yang sama. Hasil dari kedua perhitungan ini kemudian ditambahkan untuk mendapatkan keseluruhan biaya di dalam method `hitungTotalBiaya()`.

4. Perhatikan class `MainPercobaan2`. Untuk apakah sintaks `p.setMobil(m)` dan `p.setSopir(s)`?

-Sintaks ini berfungsi untuk menghubungkan objek Mobil (m) dan objek Sopir (s) yang sudah dibuat sebelumnya dengan objek Pelanggan (p). Dengan ini, atribut mobil dan sopir pada objek pelanggan p akan diisi dengan referensi ke objek m dan s tersebut.

5. Untuk apakah proses `p.hitungBiayaTotal()`?

-Proses ini digunakan untuk menghitung seluruh biaya yang harus dibayar oleh pelanggan. Method ini menjalankan kalkulasi gabungan yang menjumlahkan hasil dari biaya total mobil dan biaya total sopir

6. Pada Langkah 7, `p.getMobil().getMerk()` memanggil dua method sekaligus secara berantai. Jelaskan urutan eksekusinya: objek apa yang dikembalikan `p.getMobil()`, dan objek apa yang kemudian dipanggil `.getMerk()`-nya?

-Pertama, `p.getMobil()` dieksekusi dan mengembalikan referensi objek bertipe Mobil yang terhubung dengan pelanggan tersebut.

-Kedua, dari objek Mobil yang baru saja dikembalikan tersebut, dipanggil method `.getMerk()` yang akan mengeksekusi dan mengembalikan data String berupa merk dari mobil tersebut.

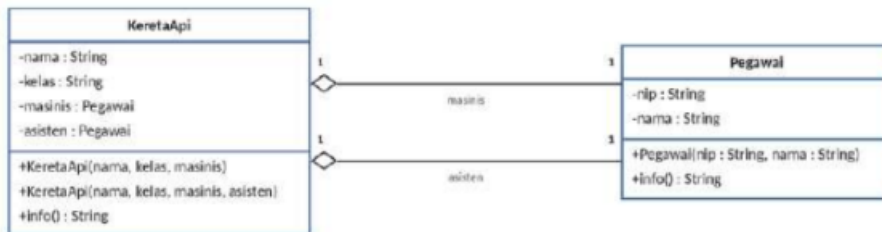
7. Andaikan `p.setMobil(m)` tidak pernah dipanggil lalu `p.hitungBiayaTotal()` dijalankan, error apa yang akan muncul? Jelaskan mengapa error itu terjadi, dikaitkan dengan konsep referensi objek yang sudah kita pelajari sebelumnya.

-Jika `p.setMobil(m)` tidak pernah dijalankan, maka atribut mobil pada objek Pelanggan tidak diinisialisasi sehingga nilai referensinya masih null. Ketika `p.hitungTotalBiaya()` dijalankan, kode akan mencoba mengeksekusi

`mobil.hitungBiayaMobil(hari)`. Karena objek mobil bernilai null, program tidak dapat menemukan referensi method yang dipanggil di dalam memori, sehingga memicu `NullPointerException`.

Percobaan 3: Aggregation dengan Dua Role ke Kelas yang Sama (Kereta Api)

Langkah 1: Diagram kelas



Langkah 2: Buat class Pegawai

```
public class Pegawai {
    private String nip;
    private String nama;

    public Pegawai(String nip, String nama){
        this.nip = nip;
        this.nama = nama;
    }

    public void setNip(String nip){
        this.nip = nip;
    }

    public String getNip(){
        return nip;
    }

    public void setNama(String nama){
        this.nama = nama;
    }

    public String getNama(){
        return nama;
    }

    public void info(){
        String info = "";
        info += "NIP: " + this.nip + "\n";
        info += "Nama: " + this.nama + "\n";
        System.out.print(info);
    }
}
```

Langkah 3: Buat class KeretaApi

```

public class KeretaApi {
    private String nama;
    private String kelas;
    private Pegawai masinis;
    private Pegawai asisten;
}

```

Langkah 4: Tambah dua constructor KeretaApi

```

public class KeretaApi {
    private String nama;
    private String kelas;
    private Pegawai masinis;
    private Pegawai asisten;

    public KeretaApi(String nama, String kelas, Pegawai masinis) {
        this.nama = nama;
        this.kelas = kelas;
        this.masinis = masinis;
    }

    public KeretaApi(String nama, String kelas, Pegawai masinis,
        Pegawai asisten) {
        this.nama = nama;
        this.kelas = kelas;
        this.masinis = masinis;
        this.asisten = asisten;
    }
}

```

Langkah 5: Tambah setter/getter dan info()

```

1 public class KeretaApi {
2     private String nama;
3     private String kelas;
4     private Pegawai masinis;
5     private Pegawai asisten;
6
7     public KeretaApi(String nama, String kelas, Pegawai masinis){
8         this.nama = nama;
9         this.kelas = kelas;
10        this.masinis = masinis;
11    }
12    public KeretaApi(String nama, String kelas, Pegawai masinis, Pegawai asisten){
13        this.nama = nama;
14        this.kelas = kelas;
15        this.masinis = masinis;
16        this.asisten = asisten;
17    }
18    public void setMasinis(Pegawai masinis){
19        this.masinis = masinis;
20    }
21    public Pegawai getMasinis(){
22        return masinis;
23    }
24    public void setAsisten(Pegawai asisten){
25        this.asisten = asisten;
26    }
27    public Pegawai getAsisten(){
28        return asisten;
29    }
30    public String info(){
31        String info = "";
32        info += "Nama: " + this.nama + "\n";
33        info += "Kelas: " + this.kelas + "\n";
34        info += "Masinis: " + this.masinis.info() + "\n";
35        info += "Asisten: " + this.asisten.info() + "\n";
36        return info;
37    }
38 }

```

Langkah 6: Uji dengan kedua atribut terisi

```
public class MainPercobaan3 {  
    Run | Debug  
    public static void main (String[] args) {  
        Pegawai masinis = new Pegawai(nip: nip: "1234", nama: nama: "Gaya Baru");  
        Pegawai asisten = new Pegawai(nip: nip: "4567", nama: nama: "Patrick Star");  
        KeretaApi keretaApi = new KeretaApi(nama: nama: "Gaya Baru");  
        System.out.println(keretaApi.info());  
    }  
}
```

```
Nama: Gaya Baru  
Kelas: Bisnis  
Masinis: NIP: 1234  
Nama: Spongebob Squarepants  
  
Asisten: NIP: 4567  
Nama: Patrick Star
```

Langkah 7: Kenali NullPointerException lebih dulu

```
public class MainPertanyaan {  
    Run | Debug  
    public static void main (String[] args) {  
        Pegawai masinis = new Pegawai(nip: nip: "1234", nama: nama: "Gaya Baru");  
        KeretaApi keretaApi = new KeretaApi(nama: nama: "Gaya Baru");  
        System.out.println(keretaApi.info());  
    }  
}
```

MainPertanyaan.java id.ac.polinema.relasiclass.percobaan3

asisten cannot be resolved to a variable Java(33554515) [Ln 4, Col 77]

Langkah 9: Perbaiki dengan guard clause

```
public String info(){  
    String info = "";  
    info += "Nama: " + this.nama + "\n";  
    info += "Kelas: " + this.kelas + "\n";  
    info += "Masinis: " + this.masinis.info() + "\n";  
    if (this.asisten != null) {  
        info += "Asisten: " + this.asisten.info() + "\n";  
    }  
    return info;  
}
```

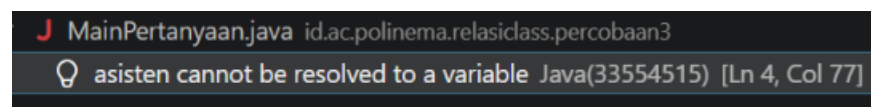
```
Nama: Gaya Baru  
Kelas: Bisnis  
Masinis: NIP: 1234  
Nama: Spongebob Squarepants
```

Pertanyaan Percobaan 3

1. Di dalam method `info()` pada class `KeretaApi`, baris `this.masinis.info()` dan `this.asisten.info()` digunakan untuk apa?

-digunakan untuk memanggil method `info()` dari instansiasi objek Pegawai yang tersimpan dalam atribut `masinis` dan `asisten`. Pemanggilan method ini berfungsi untuk mengambil data String berupa NIP dan Nama dari masing-masing pegawai, yang selanjutnya dirangkai menjadi satu kesatuan informasi data kereta api.

2. Apa hasil output dari MainPertanyaan sebelum diperbaiki (Langkah 8)? Mengapa hal tersebut dapat terjadi?



karena kelas tersebut membuat objek `KeretaApi` menggunakan constructor 3 parameter, di mana argumen untuk `asisten` tidak disertakan. Ketika method `info()` mencoba mengeksekusi `this.asisten.info()`, program gagal berjalan karena objek `asisten` belum diinisialisasi sehingga sistem mencoba mengakses method dari sebuah referensi kosong

3. Kaitkan dengan materi referensi objek: apa isi variabel `asisten` di dalam objek `KeretaApi` yang dibuat lewat constructor 3-parameter, sebelum guard clause ditambahkan?

-Isi variabel `asisten` pada objek `KeretaApi` yang dibuat melalui constructor 3 parameter adalah `null`

4. Setelah guard clause ditambahkan (Langkah 9), apakah objek `masinis` juga perlu dicek dengan cara yang sama? Perhatikan kedua constructor `KeretaApi`, apakah mungkin `masinis` bernilai `null`? Jelaskan.

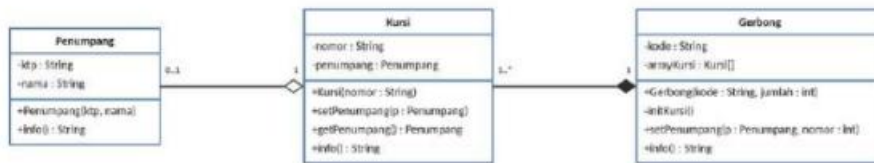
-Ya, `masinis` juga sebaiknya dicek. Meskipun kedua constructor di kelas `KeretaApi` mewajibkan kita memasukkan data `masinis`, seseorang masih bisa saja sengaja atau tidak sengaja memasukkan nilai `null` (kosong) saat membuat objeknya. Jadi, pengecekan ini tetap penting agar program tidak error jika data `masinis` ternyata kosong.

5. Kelas Pegawai dipakai lewat dua atribut berbeda (`masinis` dan `asisten`) pada `KeretaApi`. Apakah ini membuat `KeretaApi` punya dua objek Pegawai yang berbeda, atau satu objek Pegawai yang dipakai dua kali? Jelaskan berdasarkan kode pada Langkah 6.

-Kelas KeretaApi memiliki dua objek Pegawai yang berbeda. Alasannya, pada file MainPercobaan3, perintah new Pegawai() ditulis sebanyak dua kali untuk membuat data masinis dan asisten secara terpisah. Jadi, ini murni dua objek yang berdiri sendiri, bukan satu objek yang dipakai berulang.

Percobaan 4: Array of Object dan Multiplicity (Gerbong, Kursi, dan Penumpang)

Langkah 1: Diagram kelas



Langkah 2: Buat class Penumpang

```

public class Penumpang {
    private String ktp;
    private String nama;

    public Penumpang(String ktp, String nama){
        this.ktp = ktp;
        this.nama = nama;
    }

    public String getKtp() {
        return ktp;
    }

    public String getNama() {
        return nama;
    }

    public String info(){
        String info = "";
        info += "KTP: " + ktp + "\n";
        info += "Nama: " + nama + "\n";
        return info;
    }
}

```

Langkah 3: Buat class Kursi (has-a 0..1 Penumpang)

```
public class Kursi {
    private String nomor;
    private Penumpang penumpang;

    public Kursi(String nomor){
        this.nomor = nomor;
    }
    public void setPenumpang(Penumpang penumpang){
        this.penumpang = penumpang;
    }
    public Penumpang getPenumpang(){
        return penumpang;
    }
    public String info(){
        String info = "";
        info += "Nomor: " + this.nomor + "\n";
        if (this.penumpang != null) {
            info += "Penumpang: " + this.penumpang.info() +
        }
        return info;
    }
}
```

Langkah 4: Buat class Gerbong (has-a 1..* Kursi, Composition)

```
public class Gerbong {
    private String kode;
    private Kursi[] arrayKursi;

    public Gerbong(String kode, int jumlah){
        this.kode = kode;
        this.arrayKursi = new Kursi[jumlah];
        this.initKursi();
    }

    private void initKursi(){
        for (int i = 0; i < this.arrayKursi.length; i++) {
            this.arrayKursi[i] = new Kursi(String.valueOf(i
        }
    }
}
```

Langkah 5: Tambah method navigasi ke Kursi tertentu

```
public class MainPercobaan4 {  
    Run | Debug  
    public static void main(String[] args){  
        Penumpang p = new Penumpang(ktp: "12345", nama: "Mr.Krab");  
        Gerbong gerbong = new Gerbong(kode: "A", jumlah: 10);  
        gerbong.setPenumpang(nomor: 1, p);  
        System.out.println(gerbong.info());  
    }  
}
```

Langkah 6: Uji di MainPercobaan4

```
public class MainPercobaan4 {  
    Run | Debug  
    public static void main(String[] args){  
        Penumpang p = new Penumpang(ktp: "12345", nama: "Mr.Krab");  
        Gerbong gerbong = new Gerbong(kode: "A", jumlah: 10);  
        gerbong.setPenumpang(nomor: 1, p);  
        System.out.println(gerbong.info());  
    }  
}
```

```
Kode: A  
Nomor: 1  
Penumpang: KTP: 12345  
Nama: Mr.Krab
```

```
Nomor: 2  
Nomor: 3  
Nomor: 4  
Nomor: 5  
Nomor: 6  
Nomor: 7  
Nomor: 8  
Nomor: 9  
Nomor: 10
```

Pertanyaan Percobaan 4

1. Pada main program dalam class MainPercobaan4, berapakah jumlah kursi dalam Gerbong A?

-Jumlah kursi dalam Gerbong A adalah 10 kursi. Angka ini ditentukan saat membuat objek Gerbong lewat parameter kedua constructor.

2. Perhatikan potongan kode if (this.penumpang != null) { ... } pada method info() dalam class Kursi. Apa maksud kode tersebut?

untuk mengecek apakah kursi tersebut sudah diisi oleh penumpang atau belum. Jika kursi tidak kosong (penumpang != null), barulah method memanggil data

info penumpang. Ini adalah langkah aman untuk mencegah program error (Crash/NullPointerException) saat mencetak informasi pada kursi yang masih kosong.

3. Mengapa pada method setPenumpang() dalam class Gerbong, nilai nomor dikurangi dengan angka 1? 17

-Nilai nomor dikurangi 1 karena indeks array di Java selalu dimulai dari angka 0, sedangkan nomor urut kursi di kehidupan nyata (dan input pengguna) dimulai dari angka 1. Pengurangan ini memastikan kursi nomor 1 tersimpan di indeks ke-0, nomor 2 di indeks ke-1, dan seterusnya.

4. Instansiasi objek baru budi dengan tipe Penumpang, kemudian masukkan objek baru tersebut pada gerbong dengan gerbong.setPenumpang(budi, 1), menimpa Mr. Krab yang sudah duduk di sana. Apakah yang terjadi? Apakah Java memberi peringatan/error?

-Data Mr. Krab akan tertimpa oleh data Budi. Java tidak akan memberikan peringatan atau error sama sekali baik saat kompilasi maupun saat dijalankan. Secara aturan sintaks Java, proses mengganti isi sebuah variabel objek (re-assignment) dengan objek baru adalah hal yang sepenuhnya sah.

5. Modifikasi program sehingga tidak diperkenankan menduduki kursi yang sudah ada penumpang lain (tambahkan pengecekan pada Gerbong.setPenumpang() sebelum baris arrayKursi[nomor - 1].setPenumpang(...) dijalankan).

```
public void setPenumpang(int nomor, Penumpang penumpang){  
    if (this.arrayKursi[nomor - 1].getPenumpang() == null) {  
        this.arrayKursi[nomor - 1].setPenumpang(penumpang);  
    } else {  
        System.out.println("Maaf, kursi nomor " + nomor + " sudah  
    }  
}
```

6. Bandingkan tiga bentuk relasi has-a yang sudah kita praktikkan: Laptop-Processor (Percobaan 1, 1- 1), KeretaApi-Pegawai (Percobaan 3, dua relasi 1-1 bernama), dan Gerbong-Kursi (Percobaan 4, 1..*). Untuk kasus seperti apa kita akan memilih array, dan untuk kasus seperti apa kita akan memilih atribut bernama satu-satu?

Pemilihan antara array atau atribut satu per satu tergantung pada peran dan jumlah datanya:

- Pilih Array : Jika kita butuh menyimpan banyak objek yang jenis dan peran/fungsinya seragam, seperti deretan Kursi di dalam Gerbong.

- Pilih atribut bernama: Jika objek yang disimpan jumlahnya sedikit (umumnya satu atau dua) dan peran masing-masing objek sangat spesifik/berbeda, seperti masinis yang tugasnya berbeda dengan asisten di Kereta Api, atau Processor utama di dalam Laptop.

7. Terapkan kriteria kode (siapa yang memanggil new) pada dua relasi has-a di Percobaan ini: GerbongKursi dan Kursi-Penumpang. Manakah yang Aggregation dan manakah yang Composition? Tunjukkan baris kode yang menjadi bukti untuk masing-masing.

- Relasi Gerbong-Kursi adalah Composition (Kuat). Alasannya karena objek kursi secara mandiri diciptakan oleh gerbong di dalam kelasnya. Jika gerbong hancur/dihapus, kursi di dalamnya ikut hancur.

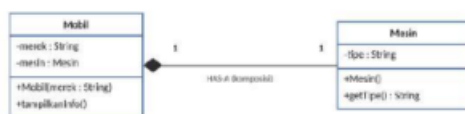
```
private void initKursi(){
    for (int i = 0; i < this.arrayKursi.length; i++) {
        this.arrayKursi[i] = new Kursi(String.valueOf(i));
    }
}
```

Relasi Kursi-Penumpang adalah Aggregation (Lemah). Alasannya karena objek penumpang diciptakan di luar, baru kemudian dimasukkan (dititipkan) ke dalam kursi. Jika kursi dihapus, objek penumpang tersebut tetap ada di memori dan bisa dipindah ke kursi lain.

```
public static void main(String[] args){
    Penumpang p = new Penumpang(ktp: "12345", nama: "Mr.Krab");
    Gerbong gerbong = new Gerbong(kode: "A", jumlah: 10);
}
```

Percobaan 5: Composition (Mobil dan Mesin)

Langkah 1: Diagram kelas



Langkah 2: Buat class Mesin

```
public class Mesin {
    private String tipe;

    public Mesin(){
        this.tipe = "4-silinder";
    }

    public String getTipe(){
        return tipe;
    }
}
```

Langkah 3: Buat class Mobil — Mesin dibuat sendiri di dalam constructor

```
public class Mobil {  
    private String merk;  
    private Mesin mesin;  
  
    public Mobil(String merk){  
        this.merk = merk;  
        this.mesin = new Mesin();  
    }  
}
```

Langkah 4: Tambah tampilkanInfo(), sengaja tanpa getter/setter Mesin

```
public class Mobil {  
    private String merk;  
    private Mesin mesin;  
  
    public Mobil(String merk){  
        this.merk = merk;  
        this.mesin = new Mesin();  
    }  
    public void tampilkanInfo(){  
        System.out.println("Mobil: " + merk);  
        System.out.println("Mesin: " + mesin.getTipe());  
    }  
}
```

Langkah 5: Uji di MainPercobaan5

```
public class MainPercobaan5 {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil = new Mobil(merk: "Avanza");  
        mobil.tampilkanInfo();  
    }  
}
```

Pertanyaan Percobaan 5

1. Pada class Mobil, baris manakah yang menunjukkan bahwa Mesin adalah bagian yang “dimiliki secara eksklusif” oleh Mobil (bukan sekadar “dipinjam”)?

```
public Mobil(String merk){  
    this.merk = merk;  
    this.mesin = new Mesin();  
}
```

2. Apa yang terjadi secara desain jika ditambahkan method `setMesin(Mesin mesin)` pada class `Mobil`? Apakah relasi ini akan tetap menjadi `Composition`? Jelaskan.

-relasi ini tidak lagi murni `Composition`, melainkan bergeser menjadi `Aggregation`. Alasannya karena keberadaan setter tersebut memungkinkan sebuah objek `Mesin` yang diciptakan secara mandiri di luar kelas `Mobil` untuk diinjeksikan ke dalam kelas tersebut. Akibatnya, siklus hidup objek `Mesin` tersebut tidak lagi bergantung sepenuhnya secara eksklusif pada objek `Mobil`.

3. Bandingkan dengan Percobaan 1 (`Laptop-Processor`): sebutkan satu perbedaan baris kode yang membuat salah satunya `Aggregation` dan yang lain `Composition`.

-Perbedaan utamanya terletak pada siapa dan di mana objek dikonstruksi menggunakan keyword `new`. Pada kasus `Composition` di kelas `Mobil`, instansiasi objek dilakukan langsung di dalam kelas pemiliknya menggunakan baris `this.mesin = new Mesin();`. Sedangkan pada `Aggregation` (seperti `Laptop-Processor`), inisialisasi objek (misalnya `new Processor()`) dilakukan secara terpisah di luar kelas utama, kemudian objek tersebut disisipkan melalui parameter constructor atau setter.

4. Jika objek mobil di `MainPercobaan5` di-set null setelah `tampilkanInfo()` dipanggil, apa yang terjadi pada objek `Mesin` miliknya? Bandingkan dengan nasib objek `Processor` pada Percobaan 1 seandainya objek `Laptop`-nya dihapus, apakah `Processor` tersebut masih bisa “diselamatkan” oleh kode lain? Kenapa `Mesin` tidak bisa?

-Jika objek mobil di file `MainPercobaan5.java` di-set menjadi null, objek `Mesin` di dalamnya juga akan ikut hancur dan dihapus dari memori. Pada Percobaan 1, objek `Processor` masih bisa diselamatkan (tetap hidup di memori) karena ia diciptakan sebagai variabel independen di luar `Laptop`. `Mesin` tidak bisa diselamatkan karena referensinya hanya ada di dalam objek `Mobil`; ketika `Mobil` hilang, satu-satunya akses menuju memori `Mesin` tersebut juga terputus.

5. Coba (secara terpisah, boleh di file/package percobaan sendiri) tambahkan constructor kedua pada `Mobil` yang menerima parameter `Mesin`, mirip pola Percobaan 1: `public Mobil(String merek, Mesin mesin) { this.merek = merek; this.mesin = mesin; }`. Kalau constructor ini yang dipakai, apakah `Mobil-Mesin` berubah menjadi `Aggregation`? Jelaskan alasannya.

```

public class MobilCoba {
    private String merk;
    private Mesin mesin;

    // Constructor 1 (Composition)
    public MobilCoba(String merk){
        this.merk = merk;
        this.mesin = new Mesin();
    }

    // Constructor 2 baru (Aggregation)
    public MobilCoba(String merk, Mesin mesin) {
        this.merk = merk;
        this.mesin = mesin;
    }
}

```

```

public class MainCoba {
    Run | Debug
    public static void main(String[] args) {
        // Mesin diciptakan sendiri secara mandiri di luar M
        Mesin mesinAvanza = new Mesin();

        // Mesin dimasukkan (diinjeksikan) ke dalam Mobil
        MobilCoba mobil = new MobilCoba(merk: "Avanza", mesi
    }
}

```

Pada kode MainPercobaan di atas, objek Mesin diciptakan secara mandiri terlebih dahulu di luar kelas Mobil. Objek Mesin tersebut hanya "dititipkan" ke dalam Mobil lewat parameter constructor. Karena siklus hidupnya kini terpisah, jika objek mobil dihapus atau di-set null, objek mesinAvanza akan tetap utuh dan hidup di dalam memori.

Percobaan 6: Dependency / Uses-A (Laptop Mencetak Dokumen ke Printer)

Langkah 1: Diagram kelas



Langkah 2: Buat class Printer

```

public class Printer {
    private String merk;

    public Printer(String merk) {
        this.merk = merk;
    }

    public void cetak(String namaFile){
        System.out.println("[ " + merk + " ] mencetak " + namaFile);
        System.out.println("[ " + merk + " ] Selesai " );
    }
}

```

Langkah 3: Buat class Laptop dengan method cetakDokumen()

```

public class Laptop {
    private String merk;

    public Laptop(String merk) {
        this.merk = merk;
    }

    public void cetakDokumen(Printer printer, String namaFile) {
        System.out.println(merk + " Mengirim dokumen ke printer.");
        printer.cetak(namaFile);
    }
}

```

Langkah 4: Uji di MainPercobaan6

```

public class MainPercobaan6 {
    Run | Debug
    public static void main(String[] args) {
        Laptop laptop = new Laptop(merk: "Thinkpad");
        Printer printer = new Printer(merk: "Epson L3110");
        laptop.cetakDokumen(printer, namaFile: "Laporan.pdf");
    }
}

```

Pertanyaan Percobaan 6

1. Apakah class Laptop pada percobaan ini memiliki atribut bertipe Printer? Bandingkan dengan Percobaan 1, di mana Processor disimpan sebagai atribut Laptop.

- Tidak, kelas Laptop tidak memiliki atribut bertipe Printer. Kelas Laptop hanya memiliki atribut merk bertipe String. Berbeda dengan Percobaan 1 di mana Processor secara permanen dijadikan bagian penyusun Laptop (disimpan sebagai atribut), di percobaan ini Printer tidak dimiliki oleh Laptop.

2. Setelah method cetakDokumen() selesai dijalankan, apakah Laptop masih menyimpan referensi ke objek printer yang tadi dipakai? Jelaskan berdasarkan baris kode class Laptop.

-Tidak. Pada kode kelas Laptop, objek printer hanya diteruskan masuk sebagai parameter di dalam method cetakDokumen(Printer printer, String namaFile). Karena ia hanya bertindak sebagai parameter, begitu proses di dalam method tersebut selesai dieksekusi, referensi ke objek printer itu akan otomatis terlepas dan Laptop tidak menyimpannya sama sekali.

3. Mengapa relasi Laptop-Printer pada percobaan ini disebut Dependency (uses-a), bukan Aggregation, meskipun sama-sama melibatkan dua objek yang saling berinteraksi?

-Disebut Dependency ("uses-a" atau menggunakan) karena Laptop hanya meminjam dan memakai Printer sebentar untuk menjalankan sebuah fungsi tertentu (mencetak), tanpa menjadikannya sebagai bagian tetap dari struktur Laptop. Jika ini adalah Aggregation ("has-a" atau memiliki), Laptop harus menjadikan Printer sebagai atribut yang menetap.

4. Coba ubah kode Laptop supaya Printer disimpan sebagai atribut (mis. private Printer printerDefault, diisi lewat constructor atau setter, lalu dipakai kembali di cetakDokumen() tanpa parameter Printer). Apakah relasi ini sekarang berubah dari Dependency menjadi Aggregation? Jelaskan.

```

public class Laptop {
    private String merk;
    // Printer sekarang disimpan sebagai atribut
    private Printer printerDefault;

    public Laptop(String merk) {
        this.merk = merk;
    }

    // Diisi lewat setter (menunjukkan Aggregation karena diisi dari
    public void setPrinter(Printer printer){
        this.printerDefault = printer;
    }

    // Parameter Printer dihilangkan, sekarang menggunakan atribut
    public void cetakDokumen(String namaFile){
        if (this.printerDefault != null) {
            System.out.println(merk + " Mengirim dokumen ke printer
            this.printerDefault.cetak(namaFile);
        } else {
            System.out.println(merk + " tidak terhubung ke printer
        }
    }
}

```

```

public class MainPercobaan6 {
    Run | Debug
    public static void main(String[] args) {
        Laptop laptop = new Laptop(merk: "Thinkpad");
        Printer printer = new Printer(merk: "Epson L3110");
        // Printer "ditanamkan" atau di-set ke dalam Laptop
        laptop.setPrinter(printer);
        laptop.cetakDokumen(namaFile: "Laporan.pdf");
    }
}

```

Dengan kode di atas, objek Laptop sekarang "memiliki" (has-a) objek Printer yang menempel pada dirinya lewat atribut printerDefault, sehingga relasinya sah berubah menjadi Aggregation.

5. Lengkapi tabel berikut dengan kata-katamu sendiri (boleh dijawab di laporan): untuk masing-masing dari Aggregation, Composition, dan Dependency, sebutkan (a) apakah objek part disimpan sebagai atribut atau tidak, dan (b) siapa yang memanggil new untuk membuat objek part tersebut.

- Aggregation:
 - (a) Objek disimpan sebagai atribut.

- (b) Perintah new dipanggil di luar kelas pemilik (objek diciptakan terpisah secara mandiri, lalu "dititipkan" ke dalam kelas pemilik lewat constructor atau setter).
- Composition:
 - (a) Objek disimpan sebagai atribut.
 - (b) Perintah new dipanggil di dalam kelas pemilik (objek diciptakan secara internal dan eksklusif oleh kelas pemiliknya).
- Dependency:
 - (a) Objek tidak disimpan sebagai atribut (hanya mampir sebagai parameter pada method).
 - (b) Perintah new dipanggil di luar kelas (objek diciptakan di luar dan hanya diteruskan saat method tertentu butuh dipanggil).

Tugas Mandiri

1. Rancang satu studi kasus sendiri (bebas topiknya, misalnya perpustakaan, klinik, toko online, dsb.), gambarkan diagram kelasnya, lalu implementasikan ke dalam program. Studi kasus wajib melibatkan minimal 4 class (class yang berisi main tidak dihitung) dan wajib mencakup ketiganya: minimal satu relasi Aggregation, satu Composition, dan satu Dependency. Tandai pada laporanmu, bagian mana dari kode yang merupakan masing-masing jenis relasi tersebut, dan sertakan alasannya.

```
public class Dokter {
    private String nama;
    private String spesialis;

    public Dokter(String nama, String spesialis) {
        this.nama = nama;
        this.spesialis = spesialis;
    }

    // --> BUKTI DEPENDENCY: Pasien hanya menjadi parameter sementara di method ini.
    // Dokter tidak menyimpan Pasien sebagai atribut tetap.
    public void periksa(Pasien pasien) {
        System.out.println("dr. " + nama + " (" + spesialis + ") sedang memeriksa " +
            pasien.getNama() + " dengan keluhan: " + pasien.getKeluhan());
    }

    public String getNama() { return nama; }
}
```

```

public class RumahSakit {
    private String namaRS;
    private RuangUGD ugd; // Atribut Composition
    private Dokter dokterJaga; // Atribut Aggregation

    public RumahSakit(String namaRS) {
        this.namaRS = namaRS;
        // --> BUKTI COMPOSITION: Objek RuangUGD diciptakan mutlak di dalam constructor RS.
        // Jika RumahSakit dihapus, RuangUGD otomatis lenyap.
        this.ugd = new RuangUGD(kapasitas: kapasitas: 20);
    }

    // --> BUKTI AGGREGATION: Objek Dokter diciptakan di luar, lalu diinjeksikan lewat setter.
    // Dokter bisa pindah atau tetap ada meskipun RumahSakit ini tutup/dihapus.
    public void setDokterJaga(Dokter dokter) {
        this.dokterJaga = dokter;
    }

    public void operasional(Pasien p) {
        System.out.println("--- " + namaRS + " ---");
        ugd.infoUGD();
        if (dokterJaga != null) {
            dokterJaga.periksa(p);
        }
    }
}

```

```

public class RuangUGD {
    private int kapasitas;

    public RuangUGD(int kapasitas) {
        this.kapasitas = kapasitas;
    }

    public void infoUGD() {
        System.out.println("UGD Siaga dengan kapasitas: " + kapasitas + " kasur.");
    }
}

```

```

public class Pasien {
    private String nama;
    private String keluhan;

    public Pasien(String nama, String keluhan) {
        this.nama = nama;
        this.keluhan = keluhan;
    }

    public String getName() {
        return nama;
    }

    public String getKeluhan() {
        return keluhan;
    }
}

```

```

public class MainRumahSakit {
    Run | Debug
    public static void main(String[] args) {
        // Objek diciptakan mandiri di luar class pemilik
        Dokter drAndi = new Dokter(nama: nama: "Andi", spesialis: spesialis: "Sp.PD");
        Pasien budi = new Pasien(nama: nama: "Budi", keluhan: keluhan: "Demam tinggi");

        // Pembuatan Rumah Sakit (otomatis membuat RuangUGD di dalamnya)
        RumahSakit rsSehat = new RumahSakit(namaRS: namaRS: "RS Sehat Selalu");

        // Memasukkan dokter ke dalam RS (Aggregation)
        rsSehat.setDokterJaga(drAndi);

        // Menjalankan operasional (Dependency Dokter -> Pasien terjadi di sini)
        rsSehat.operasional(budi);
    }
}

```

```

--- RS Sehat Selalu ---
UGD Siaga dengan kapasitas: 20 kasur.
dr. Andi (Sp.PD) sedang memeriksa Budi dengan keluhan: Demam tinggi

```

2. Jawab singkat (3-5 kalimat): dalam merancang sistem barumu sendiri, bagaimana kita memutuskan sebuah relasi antar class seharusnya Aggregation, Composition, atau Dependency? Sebutkan pertanyaan kunci yang kita ajukan ke diri sendiri saat memutuskan

1. Apakah objek bagian (part) bergantung mutlak dan akan hancur jika objek pemiliknya dihapus? Jika ya, gunakan Composition.
2. Apakah objek bagian adalah penyusun dari kelas pemilik, namun ia bisa diciptakan dan berdiri sendiri di luar sistem? Jika ya, gunakan Aggregation.
3. Apakah objek tersebut hanya dipinjam atau dipakai sementara waktu untuk menjalankan sebuah fungsi tertentu tanpa perlu disimpan sebagai data atribut? Jika ya, gunakan Dependency.